

Virtual Fit

Try it on before you buy it — browser-based, body-aware virtual fitting room.
Study notes & viva preparation handout

Student: Rukaiya Sultana · rukaiyam70@gmail.com

Live app: <https://vfit.rukaiya.tech> · **Source:** github.com/Rukaiya-Sultana/virtual-fit

Stack: Next.js 14 · React 18 · TypeScript · Prisma · Supabase (PostgreSQL) · MediaPipe WASM · Tailwind · Docker / Dokploy / AWS Lightsail

How to use this handout: Sections 1–10 are the content of the project (what it is, how every part works, and why it was built that way). Section 11 is the viva bank — 50+ likely questions grouped by topic, each with a spoken-style answer. Section 12 is a suggested live-demo script and a numbers cheat sheet.

- | | |
|--|---------------------------------|
| 1. Project at a glance | • 7. AI fashion assistant |
| 2. Tech stack & why each choice | • 8. Security measures |
| 3. Architecture & request lifecycle | • 9. API reference |
| 4. Database design | • 10. Deployment & DevOps |
| 5. The fitting engine (core algorithm) | • 11. Viva question bank (50+) |
| 6. Authentication & sessions | • 12. Demo script & cheat sheet |

1 · Project at a glance

Elevator pitch (30 seconds): "Virtual Fit is a demo clothing store with a real virtual fitting room. The shopper uploads one photo; a pose-detection model runs **entirely inside their browser** and finds 33 body landmarks; garments from the catalog are then fitted onto the detected torso geometrically, in about a second. An AI assistant takes plain-language commands like 'make it slightly oversized'. Around the fitting room there is a full storefront — accounts, cart, checkout, orders — plus an admin panel. The heavy computation happens on the client, so the whole platform runs on a tiny \$5 VPS and photos never leave the device for processing."

What is real vs. simulated

- **Real:** registration/login with hashed passwords, sessions, catalog CRUD, cart, checkout with stock checks, orders + status workflow, admin panel (garments, categories, orders, users), pose detection, fitting engine, AI assistant.
- **Simulated by design:** payment and shipping — checkout creates a real order record but no money moves. Clearly labelled as a demo store.

Key numbers (worth memorising)

Metric	Value
Pose landmarks detected	33 (MediaPipe Pose Landmarker Lite, float16, ~6 MB, self-hosted)
Garment anchors per product	8 coordinates (left/right shoulder, left/right hem — x,y each) + width/height + length factor
Database	PostgreSQL (Supabase) · 5 tables · Prisma ORM
Demo catalog	15 garments across 6 categories, procedurally drawn as SVG → rasterised to WebP with sharp
First-load JS	~87 kB shared, ~100–107 kB per route
Session cookie lifetime	Customer 14 days · Admin 8 hours
Password hashing	bcrypt, cost factor 12
Upload limits	JPEG/PNG/WebP · ≤10 MB · downscaled to ≤1400 px client-side
Retention	Temp 1 h · uploads 24 h · saved results 24 h (sweeper cleans every 15 min)
Rate limits	Admin login 8/min · orders 10/min · AI 12/min · uploads 20/min
Runtime cost	~\$5–10/month VPS (512 MB is enough) + Supabase free tier + optional free LLM key

2 · Tech stack & why each choice

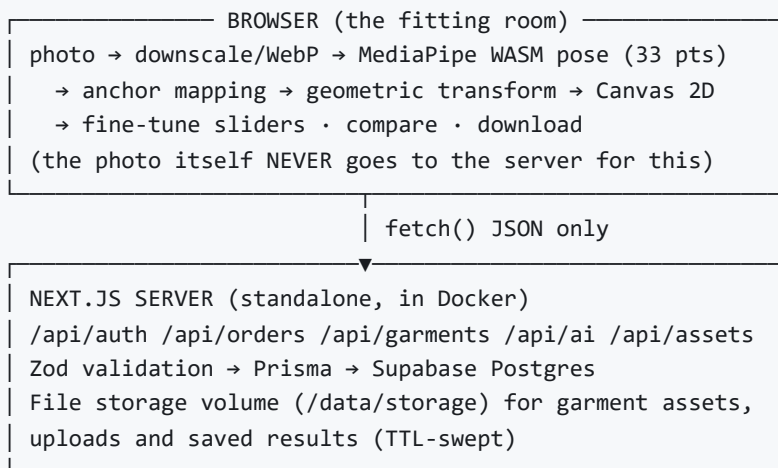
Examiners frequently ask "why technology X and not Y". This is the justification table.

Technology	Role	Why this, not the alternative
Next.js 14 (App Router)	Full-stack framework: pages + API routes in one codebase	Server components render catalog pages close to the data; API routes (<code>app/api/*</code>) keep the backend in the same repo and language. A separate Express server would mean two deployments and duplicated types. <code>output: "standalone"</code> produces a minimal container image.
React 18	UI	Server components for data pages; client components only where interactivity is needed (fitting studio, cart, admin).
TypeScript	Type safety end-to-end	Shared types (Garment, Adjustments, BodyLandmarks) flow from DB schema through API to canvas engine — refactors are caught at compile time.
Prisma	ORM / query builder	Type-safe queries generated from the schema; <code>db push</code> syncs schema to Supabase; switching SQLite→Postgres required zero query-code changes , only the datasource provider. Raw SQL would lose type safety; a heavier ORM (TypeORM) adds complexity without benefit here.
Supabase (PostgreSQL)	Managed database	Started as SQLite for zero-setup local dev; moved to managed Postgres for concurrency, backups, and a dashboard. Connection via Supabase's session pooler (port 5432) — compatible with DDL and Prisma.
MediaPipe Tasks Vision	Pose detection in-browser	Runs as WebAssembly on the client's CPU — free, offline, private. Cloud vision APIs would cost money per call and require uploading photos, violating the privacy goal. Model + WASM are self-hosted in /public (SIMD and no-SIMD builds), so there is no CDN dependency.
Tailwind CSS	Styling	Utility classes keep styling colocated; responsive variants (<code>lg:</code> , <code>sm:</code>) handled the mobile layout quickly.
sharp	Image pipeline	Rasterises the SVG garments to WebP (processed + thumbnail) at seed time; also used server-side to re-encode uploads safely.
Zod	Runtime validation	Every API route parses its input through a Zod schema — bad requests die at the boundary with a clean message, never reach Prisma.
bcryptjs	Password hashing	bcrypt with cost 12 (~250 ms/hash) — deliberately slow against brute force. Never stored or logged in plaintext.
OpenRouter (optional)	LLM provider	Free-tier models; if the key is missing the assistant falls back to a local rules engine, so the feature never breaks in a demo.
Docker + Dokploy + Lightsail	Deployment	Multi-stage Dockerfile → reproducible builds; Dokploy (self-hosted PaaS) pulls from GitHub, terminates TLS with Let's Encrypt, and redeploys on push.

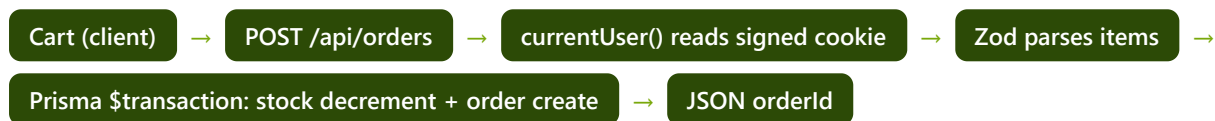
3 · Architecture & request lifecycle

3.1 The two halves of the system

The app deliberately splits work into **client-heavy ML** and **server-light commerce**:



3.2 Request lifecycle (example: placing an order)



- Stock is decremented with an **optimistic guard**: `updateMany({ where: { id, stock: { gte: qty } } })` — if another request consumed the stock in the same moment, `update.count !== 1`, the transaction aborts, and the client gets a 409 "stock changed" instead of an oversell.
- Order items are created inside the same transaction, so order + decrements succeed or fail together.

3.3 Rendering strategy

- Pages that read the database or cookies use `export const dynamic = "force-dynamic"` (home, product, account) or are dynamic by nature (cookies in `currentUser()`) — always rendered per-request, never stale.
- Pure pages (login, register, checkout shell) are static; the fitting studio is a client component.
- Static assets (pose model, WASM, garment thumbnails via `/api/assets/*`) are cacheable; a global `headers()` block sets `X-Content-Type-Options`, `X-Frame-Options: SAMEORIGIN`, `Referrer-Policy`.

3.4 Codebase layout (what lives where)

<code>src/app/</code>	pages (<code>/</code> , <code>/tryon</code> , <code>/product/[id]</code> , <code>/account</code> , <code>/admin</code> , ...)
<code>src/app/api/</code>	REST endpoints (auth, orders, garments, ai, assets, admin, health)
<code>src/lib/prisma.ts</code>	singleton PrismaClient (global in dev to survive HMR)
<code>src/lib/auth.ts</code>	session issue/verify, cookie options
<code>src/lib/admin-auth.ts</code>	admin cookie + password check
<code>src/lib/fitting/</code>	the geometry engine: anchors, transforms, canvas render
<code>src/lib/pose/</code>	MediaPipe loader + landmark post-processing
<code>src/lib/ai/</code>	assistant: LLM call, JSON schema, local rules engine, <code>apply()</code>
<code>src/lib/storage/</code>	filesystem storage adapter (<code>STORAGE_ROOT</code>)
<code>prisma/schema.prisma</code>	5 models
<code>scripts/</code>	seed, catalog import, e2e flows, cleanup, verification
<code>Dockerfile</code>	multi-stage build (deps → assets → builder → runner)

4 · Database design

4.1 Schema (Prisma → PostgreSQL)

User	id (cuid PK), name, email UNIQUE , passwordHash, role (default CUSTOMER), createdAt
Category	id PK, name UNIQUE
Product	id PK (slug e.g. "blue-hoodie"), name, description, price, stock, categoryId → Category (Restrict), color, colorHex, fit, sleeve, styleJson, occasionsJson, sizesJson (TEXT columns storing JSON arrays), formality (1-5), lengthFactor, assetPath, originalPath, thumbnailPath, leftShoulderX/Y, rightShoulderX/Y, leftHemX/Y, rightHemX/Y ← anchor geometry , width, height, badge, createdAt, updatedAt
Order	id PK, userId → User (Restrict), total, status (default PENDING), createdAt
OrderItem	id PK, orderId → Order (Cascade), productId → Product (Restrict), quantity, size, price (snapshot at purchase time)

4.2 Design decisions examiners probe

- **Why JSON strings in TEXT columns** (styleJson, occasionsJson, sizesJson)? These are display-only attributes never filtered or joined on in SQL — a dedicated table per attribute would add joins for zero query benefit. Anything we *do* query (price, category, formality) is a proper column. This is intentional denormalisation with a boundary.
- **Why price snapshot on OrderItem?** If the catalog price changes later, historical orders must still show what the customer actually paid. We copy price at checkout — classic order-line pattern.
- **onDelete behaviours:** OrderItem→Order is **Cascade** (deleting an order removes its lines — used by the admin delete feature). Order→User and OrderItem→Product are **Restrict**: you can't delete a user or product that orders reference; the admin user-delete therefore deletes the user's orders first in one transaction.
- **Order status workflow:** PENDING → CONFIRMED → PROCESSING → SHIPPED → DELIVERED, plus CANCELLED. DELIVERED and CANCELLED are terminal — the PATCH endpoint rejects further changes with 409.
- **Stock integrity:** never trust the client — quantity validated 1–10, size checked against the product's size list, stock decremented conditionally inside a transaction (see 3.2).
- **cuid primary keys:** collision-resistant, sortable, URL-safe — better than exposing sequential ints; Product additionally uses a human slug as PK.

4.3 Prisma usage specifics

- **Client singleton** (`src/lib/prisma.ts`): one PrismaClient per process, cached on `globalThis` in dev so Next.js hot-reload doesn't open endless connections.
- **Schema sync strategy:** `prisma db push` (stateless diff) instead of migration files — right-sized for a solo project; the container runs it idempotently at boot.
- **Connection string:** Supabase **session pooler** `...pooler.supabase.com:5432/postgres` — works for both DDL and runtime; the transaction pooler (6543) would require pgbouncer workarounds for schema sync.
- **Seed pipeline:** `seed-garments.mjs` draws SVG garments → sharp rasterises to WebP + catalog.json (with anchors derived from the SVG coordinates) → `migrate-catalog-to-db.mjs` upserts into Postgres. Safe to re-run (upserts, IDs stable).

5 · The fitting engine (core algorithm)

This is the heart of the project — expect detailed questions here.

5.1 Step 1 — Body detection (client, WASM)

- Photo is downscaled to ≤ 1400 px and re-encoded to WebP **in the browser** before anything else — small payloads, fast decode.
- MediaPipe **Pose Landmarker Lite** runs on WebAssembly and returns **33 landmarks** with x, y (normalised 0–1) and **visibility scores**.
- **Quality gating**: shoulders must be clearly visible; if hips are below a 0.5 visibility threshold the engine *imputes them from shoulder span* (standard body proportion) and the UI honestly labels the result "Hips estimated" — better to proceed with a flag than to fail.
- Failures are user-actionable: "face the camera", "shoulders inside frame" — the detection error carries a hint string, not a stack trace.

5.2 Step 2 — Garment anchor mapping

- Every product row stores its **own anchor coordinates** (normalised): left/right shoulder and left/right hem points — captured from the garment's original drawing geometry, so the mapping is pixel-accurate rather than guessed.
- The engine computes the user's **shoulder span** and **torso length** from the landmarks, then solves a transform that maps the garment's anchor quad onto the detected body quad: scale from shoulder ratio, vertical stretch from a per-garment `LengthFactor`, plus skew/rotation for perspective.

5.3 Step 3 — Interactive composite

- Rendered on a **Canvas 2D** with high-quality image smoothing; the garment image is drawn transformed over the photo.
- Fine-tune controls adjust scale, X/Y offset, length and rotation — a typed `Adjustments` object applied in the render loop; pointer drag pans (touch-action disabled on the canvas so mobile drags don't scroll the page).
- **Compare mode** re-renders two garments on the same body side by side; before/after toggles the raw photo; a skeleton overlay can visualise the landmarks for the demo.

Why geometric, not generative (the #1 viva question): photorealistic virtual try-on needs diffusion models on GPUs and uploads the user's photo to a server — incompatible with privacy-first, \$0-inference and offline goals. Anchor geometry is deterministic, instant (~ 1 s), explainable math (no black box), works on any CPU via WASM, and the trade-off (no fabric drape simulation) is stated openly.

5.4 Why Canvas 2D and not WebGL/three.js

We are compositing one transformed bitmap over one photo — a 2D affine-ish draw. WebGL would add a 3D engine dependency ($\sim$ hundreds of kB) for zero visual gain here. If we later simulate drape or lighting, WebGL/WebGPU becomes the right tool.

5.5 Why not GPT-4o / Gemini image generation for the try-on?

Multimodal LLMs *can* generate a photorealistic try-on picture, so examiners will ask why we don't. The answer is primarily **economic**, and secondarily about privacy, latency and trust. This is the strongest business

argument in the project — learn the table.

	Generative API (GPT-4o/5 image, Gemini image, try-on SaaS)	Virtual Fit (on-device geometry)
Cost per try-on	~\$0.02–\$0.20 per generated image (2026 list pricing; try-on SaaS charges more)	\$0.00 — runs on the shopper's own CPU
Cost at scale	1,000 try-ons/day ⇒ roughly \$600–\$6,000/month , growing linearly with success	\$0 — flat ~\$5–10/month VPS + free-tier DB at any volume
Cost structure	Pure variable cost — the more customers use it, the more you pay; marketing spikes = surprise bills	Zero marginal cost — inference is delegated to hardware the customer already owns
Setup for an SME	API keys, billing account, rate limits, content-moderation review, vendor contract	One Docker container, no keys, no billing integration
Privacy / compliance	Body photo uploaded to a third party — GDPR/data-processing agreements, AI-training data policies	Photo never leaves the browser ; nothing to disclose, nothing to leak
Latency	Upload photo + queue + generate + download ⇒ several seconds, needs internet	~1 second, local; works offline after first load
Trust / consistency	Generative models can subtly alter the face, body or garment colour — unacceptable when the question is "will <i>this</i> fit <i>me</i> ?"	Deterministic: same body + same garment ⇒ identical fit, every time
Vendor risk	Provider can change price, model behaviour or deprecate endpoints overnight	Model file (~6 MB) is pinned in our repo — self-hosted, unchangeable by anyone else

The SME economics, spelled out (memorise this): An SME's software cost = fixed + marginal. A generative API turns try-on into a **variable cost that scales with success** — every new customer using the feature eats into the very conversion gains it creates, and one viral day produces a surprise bill. Virtual Fit moves the inference to the customer's device — hardware they already own and paid for — so the store's **marginal cost per try-on is exactly zero**, and total platform cost stays flat (~\$5–10/month) from the first customer to the ten-thousandth. No API key to leak, no per-image billing, no rate-limit outage during a campaign. For a small store with thin fashion-retail margins, that is the difference between a feature that pays for itself and one that becomes a liability.

Where generative try-on *would* make sense: as an optional, clearly-labelled premium tier ("photoreal preview") for shoppers who explicitly consent to uploading a photo — layered *on top of* the free default. The architecture supports adding it behind a flag later; the free, private, instant geometric fitting remains the baseline for everyone. We already apply this "LLM where it's cheap" principle: the project **does** use an LLM for the text assistant (free tier + local fallback) — i.e., exactly where it adds value at ~zero cost, and never where it bills per body.

6 · Authentication & sessions

6.1 Register / login flow

- Zod validates input (email format, password ≥ 8 chars).

- Duplicate email → 409; wrong credentials → a **generic 401 "Email or password is incorrect"** (never reveal which field was wrong — prevents account enumeration).
- Password compared with `bcrypt.compare`; login adds a small constant delay on failure to blunt timing/brute-force, plus rate limiting (8 admin attempts/min).

6.2 Stateless signed-cookie sessions

```
token = base64url(JSON {id, role, exp}) + "." + HMAC-SHA256(payload, AUTH_SECRET)
```

- Verification: recompute the HMAC and compare with **timingSafeEqual**, check expiry, then load the user from Postgres (so deactivated users vanish even with a valid token).
- **Why stateless?** No session table, no extra DB read per static check; the only DB hit is the user fetch we want anyway. A database-session store would be the choice if we needed instant server-side revocation for many sessions.
- Cookie flags: `httpOnly` (JS can't read it → XSS can't steal it), `SameSite=Lax` (blocks cross-site CSRF sends for POST), `path=/`, 14-day max-age.
- **Secure flag is protocol-aware:** set only when the request actually arrived over HTTPS (`x-forwarded-proto` behind the proxy). Hard-coding Secure broke login on plain HTTP during development — browsers silently drop Secure cookies on http:// — a real bug found and fixed, great viva story.

6.3 Admin authentication

- Separate, minimal scheme: single admin password from env (`ADMIN_PASSWORD`, ≥ 8 chars enforced) → the cookie stores only an **HMAC derived from the password**, never the password itself; 8-hour expiry.
- Every `/api/admin/*` route calls `isAdminRequest()` first → 401 otherwise. Admin features: garment upload, categories CRUD, order status + delete, user list + delete (which cascades their orders).

7 · AI fashion assistant

7.1 Dual-engine design

- **Request:** message + last 6 chat turns (content clipped to 300 chars) + currently selected garment → `POST /api/ai` with a 25 s abort timeout and rate limit (12/min per client).
- **LLM path (if OPENROUTER_API_KEY set):** the model is prompted to answer with a strict JSON command object. The response is parsed and **validated with Zod** against a fixed action vocabulary — `adjust_fit`, `select_garment`, `recommend`, `compare`, `explain`.
- **Local rules engine (fallback, always available):** deterministic intent matching — "bigger/oversized" widens scale, "shorter" cuts length, colours and occasions map to catalog filters. No network, no key, instant. The health endpoint reports which engine is active.
- **Apply pipeline:** commands go through one typed `apply()` function that clamps values and updates the studio state; the UI then shows an "Applied: ..." note. **The model can only trigger actions the UI already allows** — it cannot inject HTML, touch the DOM, or exceed adjustment limits.

Privacy note for viva: the assistant operates on **catalog data and chat text only** — the user's photo never goes to the LLM.

8 · Security measures (checklist)

Area	Measure
Passwords	bcrypt cost 12; no plaintext ever stored or logged
Sessions	HMAC-SHA256 signed, httpOnly, SameSite=Lax, protocol-aware Secure, timing-safe comparison
Input	Zod schema validation on every API route (body size and field limits included)
Abuse	In-memory sliding rate-limit buckets keyed per IP+route (login 8, orders 10, AI 12, uploads 20 per minute)
Uploads	MIME whitelist (JPEG/PNG/WebP), 10 MB cap, client-side downscale, server re-encode with sharp, max pixel dimension
File serving	<code>/api/assets/[...path]</code> resolves and whitelists paths under STORAGE_ROOT — no path traversal; content types set explicitly
Headers	nosniff, X-Frame-Options SAMEORIGIN, strict Referrer-Policy globally
Secrets	All in env vars (.env gitignored, verified not in repo history); admin password minimum length enforced at runtime
Data retention	Temp 1 h / uploads 24 h / results 24 h TTLs with a background sweeper ($\leq$ every 15 min) — stored surface stays small
Camera/permissions	Only requested on user action; graceful messaging on HTTP where browsers block getUserMedia

9 • API reference

Endpoint	Method	Auth	Purpose
/api/health	GET	—	Status: storage writable, garment count, AI engine mode (used by Docker healthcheck)
/api/auth/register	POST	—	Create account (Zod + bcrypt) and set session cookie
/api/auth/login	POST	—	Verify credentials, set session cookie
/api/auth/logout	POST	cookie	Clear session cookie
/api/auth/me	GET	cookie	Current user (drives the navbar state)
/api/garments	GET	—	Catalog listing for shop + fitting room
/api/orders	POST	cookie	Checkout: validate items/sizes/stock → transaction → create order
/api/orders/[id]	GET	cookie	One order (ownership enforced)
/api/sessions/[id]/photo results	POST/GET	session	Opt-in saved try-on results (TTL)
/api/assets/[...path]	GET	—	Whitelisted asset serving from the storage volume
/api/ai	POST	—	Assistant: LLM or local rules; Zod-validated commands
/api/admin/login	POST/DELETE	password	Admin session cookie in/out (rate limited)
/api/admin/session	GET	admin	Admin auth probe
/api/admin/garments	POST	admin	Upload garment image + metadata (sharp re-encode)
/api/admin/categories	GET/POST/DELETE	admin	Categories (delete blocked while products exist)
/api/admin/orders	GET	admin	All orders with user + items
/api/admin/orders/[id]	PATCH/DELETE	admin	Change status (terminal states 409) / delete order (items cascade)
/api/admin/users	GET	admin	Users with order counts
/api/admin/users/[id]	DELETE	admin	Delete user + their orders in one transaction

10 · Deployment & DevOps

10.1 The Docker image (multi-stage)

```
deps      npm ci (postinstall copies MediaPipe WASM + downloads the pose model)
assets    prisma generate + draw demo garments (SVG→WebP) + catalog.json
builder   prisma generate + next build (no DB needed at build: all DB pages are dynamic)
runner    node:20-alpine + standalone server.js + prisma CLI for boot-time db push
```

- **Standalone output** ships only traced production node_modules — small image, fast cold start (server ready in ~40 ms).
- **Entrypoint self-heals:** validates DATABASE_URL → seeds the demo garment assets into the persistent volume on first boot → `prisma db push` (idempotent schema sync) → imports the demo catalog only if the product table is empty → starts the server. A broken/empty database recovers automatically.
- **Persistent state:** only `/data` (garment assets, uploads, results). The database is fully managed by Supabase — daily backups, no volume coupling.
- Container healthcheck hits `/api/health`.

10.2 The chain to production

GitHub push → **Dokploy** (on an AWS Lightsail VPS) builds the Dockerfile → runs it behind its Traefik proxy → **Let's Encrypt** certificate for HTTPS → **Cloudflare DNS** (DNS-only) points the domain. HTTPS matters twice: browsers block the live-camera API on plain HTTP, and cookies/camera/security all become strict automatically when the request is secure (the app detects this per-request).

11 · Viva question bank

Answers written the way you'd say them. Practise the bolded keywords.

A · Architecture & design decisions

A1. Explain your overall architecture in one minute.

It's a Next.js full-stack app split into two halves. The fitting room is client-heavy: MediaPipe pose detection runs as WebAssembly in the browser, and the garment is composited on a canvas with a geometric transform — so the server never processes photos. The commerce half is server-light: REST API routes validate with Zod and talk to Supabase Postgres through Prisma. The whole thing runs in a Docker container on a small VPS, with only garment assets and opt-in saved results on a persistent volume.

A2. Why is the ML on the client rather than the server?

Three reasons: privacy — the photo never leaves the device; cost — inference scales with visitors for free instead of per-request GPU billing; and offline capability, since the model is self-hosted. The trade-off is a ~6 MB one-time model download, cached by the browser.

A3. Why Next.js instead of a React SPA + separate Express API?

One codebase, one language, one deploy. Server components render data-driven pages next to the database instead of shipping JSON and rendering on the client, and API routes give me the backend without a second server. The standalone output mode also produces a minimal Docker image.

A4. Which pages are server-rendered and which are client-side, and why?

Data pages — home, product, account, orders — are dynamic server components because they read cookies or the database per request; I mark them force-dynamic so nothing stale is cached. The fitting studio, cart, checkout and admin are client components because they're interactive state machines. Login/register are static shells.

A5. How do client and server share types?

TypeScript all the way through — a Garment type used by the canvas engine is the same type the API serialises, and Prisma generates its model types from the schema. Zod schemas additionally give me runtime validation at the boundary, where compile-time types stop existing.

A6. What's the hardest technical problem you solved?

Making the garment sit convincingly on different bodies. Each garment stores its own anchor coordinates — shoulders and hems — taken from its drawing geometry, and the engine maps that quad onto the detected landmarks using shoulder span for scale and torso length with a per-garment length factor. Getting that stable without visual artefacts, across camera angles, took the most iteration.

A7. Second hardest?

Production runtime behaviour. Cookies marked Secure are silently dropped by browsers on plain HTTP — login worked locally and failed on the VPS until I made the Secure flag follow the actual request protocol from x-forwarded-proto. The same HTTPS rule blocks the camera API, so I added a native mobile camera capture fallback that works even over HTTP.

A8. Why Canvas 2D and not WebGL?

The render is one transformed bitmap over one photo — a 2D draw with image smoothing. WebGL or three.js would add a heavy dependency for no visual gain. If I were simulating fabric drape or lighting, WebGL becomes the right choice.

A9. Where does business logic live?

In the API routes and lib modules — pricing, stock checks, order creation, auth. The client renders state and can't bypass validation: quantity, size and stock are all re-checked server-side at checkout.

A10. Is it real-time? What's the latency?

Photo-to-fitted-result is about one second, all on-device: WASM inference plus a canvas draw. Fine-tune adjustments re-render within a frame because it's the same local bitmap transform.

B · Code & implementation

B1. How does the pose detection actually work?

MediaPipe's Pose Landmarker returns 33 normalised landmarks with visibility scores. I gate quality on the scores — shoulders must be visible; if hips score below 0.5 I estimate them from shoulder span using body proportions and label the result "hips estimated" so the user knows. Bad photos get actionable hints like "keep shoulders inside the frame".

B2. Walk me through the fitting transform.

The garment row stores normalised anchor points for both shoulders and both hems plus a length factor. I compute the user's shoulder span from the landmarks, derive the scale from the ratio of their span to the garment's anchor span, stretch vertically by torso length times the length factor, and add skew and rotation for perspective. The result is drawn with canvas transform onto the photo, and the user can fine-tune scale, offset, length and rotation.

B3. How is state managed in the studio?

Local React state plus refs — the canvas node, the detected landmarks, the selected garment and an Adjustments object. Adjustments are also mirrored into a ref so the AI command pipeline can read and patch them outside the render cycle.

B4. How do you prevent overselling stock under concurrency?

Inside a Prisma transaction, stock is decremented with a conditional updateMany — where id matches AND stock $\geq$ quantity. If the count of updated rows isn't exactly one, another request took the stock first, the transaction aborts and the client gets a 409 "stock changed" message. No read-then-write race.

B5. How are uploads made safe?

Client-side: type from a whitelist, size capped at 10 MB, downscaled to 1400 px and re-encoded — which also strips any embedded payload. Server-side, saved images are re-encoded again with sharp, stored under a generated session directory, and served back only through a route that whitelists resolved paths under the storage root, so directory traversal can't reach arbitrary files.

B6. How does rate limiting work, and its limits?

A small in-memory bucket keyed by client IP and route — e.g. 8 admin logins, 12 AI calls per minute. It's per-process, so with multiple replicas each instance would count separately; the next step would be a shared store like Redis. For this scale it's the right amount of engineering.

B7. Why Zod when you already have TypeScript?

TypeScript vanishes at runtime — fetch bodies are untrusted network input. Zod parses and constrains it at the boundary, rejects bad requests with clean messages, and infers the static type, so one schema covers both worlds.

B8. What happens when the browser doesn't support WASM/SIMD?

The MediaPipe package ships SIMD and non-SIMD builds and both are self-hosted, so older CPUs get the fallback build. If the model can't load at all, the UI explains it and the store part of the app keeps working — the try-on is a feature, not a dependency, of the storefront.

B9. How did you handle mobile?

Responsive Tailwind layouts; the studio reflows from three columns to a scrollable stack; the AI chat becomes a floating button with a bottom sheet; canvas drag uses pointer events with touch-action disabled; and photo capture offers the native camera input, which works even where live camera preview is blocked.

B10. What did you automate in scripts?

Seeding — SVG garments rasterised by sharp with anchors derived from the same coordinates; a catalog importer that upserts safely; end-to-end flow scripts (register → cart → order); a catalog validator and asset integrity checker; a cleanup sweeper for expired files; and a selftest endpoint.

C · Database

C1. Describe your schema.

Five tables: User, Category, Product, Order, OrderItem. Product carries commerce fields plus fitting metadata — eight anchor coordinates, dimensions and a length factor. OrderItem snapshots the price at purchase. Uniques on user email and category name; foreign keys with deliberate cascade rules — order items cascade with their order, while users and products are restricted while referenced.

C2. Why PostgreSQL over MongoDB?

The data is inherently relational — orders reference users and products, and integrity matters (restrict rules prevent deleting a product with order history). Postgres gives constraints, transactions and joins; the few document-ish fields like style tags are stored as JSON text in columns because they're only ever displayed, never queried.

C3. Why store JSON in TEXT columns — isn't that an anti-pattern?

It would be, if I queried it. Styles, occasions and size lists are display-only, so a normalised table would cost joins for zero benefit. Anything filtered or aggregated — price, category, formality, stock — is a real column. PostgreSQL also has JSONB if these ever need querying; the boundary is deliberate.

C4. Why Prisma over raw SQL or another ORM?

Generated, type-safe queries from one schema file, readable migration story with db push, and portability — I moved from SQLite to Postgres by changing one provider line. Raw SQL would lose compile-time safety; heavier ORMs add ceremony this project doesn't need.

C5. How do you handle database connections in serverless-ish environments?

A singleton client per process cached on globalThis, and I connect through Supabase's session pooler, which multiplexes connections safely. The transaction-mode pooler would need pgbouncer workarounds for DDL, so the session pooler keeps schema sync and runtime on one URL.

C6. How do migrations work?

I use prisma db push: it diffs the schema against the live database and applies the change idempotently. It suits a solo project — the container literally runs it at boot and no-ops when in sync. For a team I'd switch to versioned migrations for reviewable, replayable history.

C7. Why does OrderItem copy the price?

Historical accuracy — if a product's price changes tomorrow, past orders must still show what was paid. The line item is an immutable record; the catalog is live data. Same reason I snapshot size too.

C8. How would you find the top-selling products?

Aggregate OrderItem grouped by productId with a sum of quantity, ordered descending, joined to Product for names — one Prisma groupBy or a short SQL query. It's not exposed in the admin UI yet; it's on the roadmap.

C9. What indexes exist and why?

Primary keys on all ids, plus unique indexes on user email (login lookup and duplicate prevention) and category name. Foreign keys are indexed by default in Postgres, which covers the order→user and item→product joins. For this data size that's sufficient; a search feature would add a trigram index on product name.

C10. How is the database seeded and reset?

A seed script draws the demo garments, rasterises them with sharp, writes catalog.json with anchors from the drawing coordinates, and an importer upserts products and categories — safe to rerun, stable IDs. In production, the container imports the catalog automatically when it finds an empty product table.

D · Security

D1. How are passwords stored?

bcrypt with cost factor 12 — about a quarter second per hash, which is invisible to a user logging in but devastating to brute force. Verification uses the constant-time compare built into bcrypt.

D2. Explain your session mechanism. Why not JWT?

It's close to a JWT in spirit — a signed, stateless token: a base64 payload of user id, role and expiry, signed with HMAC-SHA256 using a server secret, stored in an httpOnly cookie. Compared to a standard JWT I skip the header and use symmetric signing only. The signature is verified with a timing-safe comparison, the expiry is checked, and the user is fetched from the database so deleted users lose access immediately.

D3. How do you prevent XSS?

React escapes all interpolated content by default; I never use dangerouslySetInnerHTML. The session cookie is httpOnly so even an injected script couldn't read it. Security headers — nosniff, frame-options, referrer policy — add another layer.

D4. How do you prevent CSRF?

Cookies are SameSite=Lax, so cross-origin sites can't trigger authenticated POSTs from the browser. State-changing endpoints are POST/PATCH/DELETE only, and there's no JSONP or wildcard CORS — the AI endpoint optionally restricts origins via an allowlist env var.

D5. What about SQL injection?

Prisma parameterises every query — user input is bound, never concatenated. Raw queries use the tagged template API which also binds parameters.

D6. Path traversal?

The asset route takes a catch-all path, resolves it, and only serves files that resolve inside the whitelisted storage directories. Anything escaping the root is rejected.

D7. How is the admin area protected?

A separate password from the environment, minimum length enforced, rate-limited to 8 attempts per minute with a constant-ish delay on failure. The cookie stores only an HMAC derived from the password — forgery without the secret is impractical — and every admin API route re-checks it server-side. There's no hidden client-side "protection"; the UI hiding is convenience, the API guard is the security.

D8. What data does the server ever see of the user's body?

Only what the user explicitly saves: an optional saved result image, TTL 24 hours and swept by the cleanup job. Detection and rendering happen locally; the AI assistant receives catalog data and chat text, never the photo.

E · AI / ML

E1. Which model, and why that one?

MediaPipe Pose Landmarker Lite — the lightest of Google's pose models: ~6 MB, float16, designed for on-device CPU inference via WASM. Heavy models buy accuracy I don't need for shoulder-to-hip garment mapping, at the cost of download size and frame latency.

E2. How does the AI assistant avoid doing something dangerous?

The LLM doesn't return free-form actions — it must answer in a JSON schema that Zod validates against a fixed vocabulary: adjust fit, select garment, recommend, compare, explain. Those map to a typed apply() function that clamps values. Worst case from a hallucination is a wrong-but-safe adjustment; the model literally cannot express anything outside the UI's capabilities.

E3. What if the LLM is down or the key is missing?

A local rules engine handles the same intents deterministically — keyword matching for bigger, smaller, up, down, colours, occasions. The health endpoint reports which engine is active, and the badge in the chat shows it. Demos never break.

E4. How would you improve the try-on realism?

In order of cost: per-size garment variants and fabric opacity shading on the current geometry; segmentation masking so garments tuck behind arms using a body-segmentation model (also MediaPipe, also on-device); and full generative try-on with a diffusion model, which would force a server GPU and photo uploads — a privacy trade I'd make opt-in, not default.

E5. What confidence thresholds do you use?

Landmark visibility of 0.5 on hips — below that I impute from shoulder span and flag it; shoulders below threshold fail with guidance instead of producing a bad fit. The overall detection confidence is surfaced in the UI as a percentage.

E6. Why not use GPT-4o or Gemini for the try-on itself?

They can generate a convincing try-on image — but they're the wrong default, especially for small businesses. First, economics: generative image APIs charge per image, a few cents each, so a store doing a thousand try-ons a day is paying hundreds to thousands of dollars a month, forever, and the bill grows linearly with the feature's success. My design runs inference on the shopper's own device — a 6 MB WASM model — so the store's marginal cost per try-on is exactly zero; the entire platform stays on a five-dollar VPS no matter the volume. Second, privacy: the body photo would be uploaded to a third-party cloud with all the GDPR and data-processing baggage that entails, while mine never leaves the browser. Third, latency and offline: seconds of round-trip versus about one second locally. Fourth, trust: a generative model can subtly change the person's face or the garment's colour — fatal when the question is "will this fit me" — whereas my geometric fit is deterministic and clamped. And note we're not anti-LLM: the text assistant does use one, on a free tier with a local fallback — the principle is "LLM where it's cheap, never where it bills per body."

E7. So is a photorealistic try-on impossible in your approach?

Not impossible — it's a deliberate trade for the default experience. The roadmap adds segmentation masking and shading for realism on-device, and a generative "photoreal preview" could be an opt-in premium tier where the user consciously uploads a photo. The key design point is that the free, private, instant path remains the baseline — generative features ride on top, they don't replace it.

F · Deployment, performance & testing

F1. Describe your deployment pipeline.

Push to GitHub → Dokploy on a Lightsail VPS builds the multi-stage Dockerfile → the entrypoint validates the database URL, seeds garment assets to the volume on first boot, syncs the Prisma schema, imports the demo catalog if the table is empty, and starts the standalone Next.js server behind Traefik with a Let's Encrypt certificate. Cloudflare handles DNS.

F2. What happens on redeploy — is data lost?

No. The database is managed by Supabase, outside the container. Garment assets and saved results live on a mounted volume at /data. The entrypoint only ever seeds when it finds empties, so redeployments are non-destructive.

F3. How did you make the container robust?

It self-heals: schema sync is idempotent, an empty or broken database gets re-seeded automatically, env values pasted with quotes are sanitised, and if the database can't initialise it fails loudly with instructions instead of serving a broken app. There's a health endpoint the platform polls.

F4. What breaks first at scale, and what do you do?

Because compute is client-side, the server only serves the app and catalog. Order: storage volume I/O — move assets to object storage like Supabase Storage or S3; then Postgres connections — the pooler plus a connection limit, then read replicas; then the Node process — horizontal replicas behind the proxy, which is also when rate limiting would move to Redis. None of these are architectural rewrites.

F5. How did you test?

Layered: Zod contract tests at the API boundary; end-to-end scripts driving register → cart → order flows against a running instance; a catalog validator and asset integrity checker; a selftest endpoint; and manual verification of the full Docker path, including deliberately breaking the database to prove auto-recovery.

F6. What's not tested that you'd test with more time?

Unit tests on the geometry maths with recorded landmark fixtures, cross-browser visual regression on the canvas output, and load testing the API. The fitting transform deserves golden-file tests since it's pure math — very testable.

F7. What are the project's limitations?

Honest list: no fabric drape simulation; fit is proportional, not size-predicting; the demo catalog is procedurally drawn rather than photographed garments; rate limiting is per-process; payments are simulated; and pose accuracy degrades with extreme camera angles — mitigated by the visibility gating and honest UX flags.

F8. What would you do differently?

Three things: versioned migrations from day one for reviewable history; unit tests on the transform math from the first commit; and structured logging with request ids — console logs were enough, but observability pays off the moment something breaks in production.

G · Product & soft questions

G1. Who is this for?

Small independent stores that can't afford enterprise try-on SaaS, plus privacy-conscious shoppers. The store gets a differentiating feature with no per-try-on cost; the shopper gets a better size decision without uploading their body to anyone.

G2. How does it make money?

As a project it doesn't — but as a product: a hosted version for small merchants per month, or self-hosted with a support tier. The unit economics work precisely because inference is free on the client.

G3. What did you learn?

That "works on my machine" fails at the boundary — real lessons were browser security rules (Secure cookies, HTTPS-only camera), proxy headers, database pooling, and designing fallbacks so a demo can never fully break. Also: keeping ML on the client turns a cost problem into a distribution feature.

12 · Demo script & cheat sheet

Suggested 5-minute live demo (in this order)

1. **Storefront** (30 s) — home grid, open a product page, note price/fit/sizes.
2. **Fitting room** (2 min) — upload/camera a photo → detection runs (point out it's local: no network tab activity!) → pick a garment → fine-tune sliders → before/after → compare two garments.
3. **AI assistant** (1 min) — type "make it slightly oversized" → watch adjustment apply; ask "something casual for university" → recommendations as clickable chips.

4. **Purchase loop** (1 min) — add to cart from the studio → register → checkout → orders page shows the order.
5. **Admin** (30 s) — login, change the order status, show users/orders management.

Three mic-drop moments if asked "show me something clever":

1. Open DevTools → Network, reload the fitting room, process a photo — **zero photo upload requests**.
2. Delete the database rows (or ask about it) → container re-seeds itself on next boot.
3. Revoke the AI key (or go offline) → assistant still works via the local rules engine.
4. Asked "why not GPT/Gemini?" → flip to the \$5.5 cost table: **\$0 marginal cost per try-on** vs \$600–\$6,000/month at scale.

One-line definitions to memorise

- **Anchor mapping:** fitting a garment by mapping its stored shoulder/hem coordinates onto the same landmarks detected on the user's body.
- **WASM:** WebAssembly — a binary instruction format that runs near-native speed in the browser; how MediaPipe executes on-device.
- **Stateless session:** authenticity proven by a signature over the data instead of a server-side session table.
- **Pooled connection:** many clients share a small set of database connections via a pooler, avoiding connection exhaustion.
- **Optimistic stock guard:** conditional decrement that fails safely if the row changed concurrently.
- **Zod validation:** runtime schema parsing of untrusted input at the API boundary.

Generated for Rukaiya Sultana · Virtual Fit · 2026 — matches the codebase as of commit c42adb0. Live: <https://vfit.rukaiya.tech> · Repo: github.com/Rukaiya-Sultana/virtual-fit